

Chapter 7: Classes Part 1

C# is an object-oriented programming language. This means that all entities are described via objects or classes. Classes in turn have characteristics known as properties and behaviors known as methods.

Classes may inherit their properties and methods from other more simple classes. For example, let's say you have a person class with properties of age, weight, height and eye color. You then want to create a student class. Just have it inherit the person class and then add in the properties specific to being a student. The same is true for methods. They are inherited from simple to more complex class types.

Well with that background in mind let's jump to the inside and see some class examples.

We start of course with a new console application named Chapter 7 Classes Part 1.

Our first example is very simple. We will define a Cube class as shown here. (Figure 7.1)

The keyword `class` is followed by the class name, in our case `Cube` with an uppercase `C`. We then define three private double data type variables named `l`, `w` and `h`. Because they are defined as private, they may only be used within this class definition.

We next define three properties of our `Cube` class. Remember, properties are characteristics of a class. The `public void setLength(double length)` syntax says create a subroutine which accepts values from the outside (public), returns no value (void), is named `setLength` and accepts one input parameter of type double named `length`. The inside of our subroutine simply takes that `length` value input from the outside and assigns it to our internal variable `l`. The same exact thing happens with the `setHeight` and `setWidth` subroutines.

The `public double getVolume()` syntax says create a subroutine which is available to the outside (public) which returns a double value and is named `getVolume`. Inside that routine we simply return the product of `l * w * h`.

Figure 7.1: Cube class Definition

```
class Cube
{
    private double l, w, h;

    0 references
    public void setLength(double length) { this.l = length; }

    0 references
    public void setWidth(double width) { this.w = width; }

    0 references
    public void setHeight(double height) { this.h = height; }

    0 references
    public double getVolume() { return l * w * h; }
}
```

In our next section of code, we start to use our new Cube class. (Figure 7.2) We start by referencing the Program class. The Program class always contains executable code. Inside the Program class we create the Main block of code. The Main block of code is where execution begins.

Execution begins with the instantiation of a Cube object with the `Cube cube1 = new Cube()` syntax. This syntax says I am going to instantiate a Cube object named `cube1` using the constructor method named the same as my class type, i.e. `Cube`. We then create a second instance of our Cube class named `cube2`.

Our code next calls the `setLength`, `setHeight` and `setWidth` methods of our Cube class using different values for our `cube1` and `cube2` objects. Finally, we call the `getVolume` method of our Cube class to display the relative volumes of our two cubes. Cute little starting example.

Figure 7.2: Cube class Main

```
class Cube
{
    private double l, w, h;

    2 references
    public void setLength(double length) { this.l = length; }

    2 references
    public void setWidth(double width) { this.w = width; }

    2 references
    public void setHeight(double height) { this.h = height; }

    2 references
    public double getVolume() { return l * w * h; }
}

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Cube cube1 = new Cube();
        Cube cube2 = new Cube();

        cube1.setHeight(2);
        cube1.setWidth(5.5);
        cube1.setLength(10);

        cube2.setHeight(4);
        cube2.setWidth(5.5);
        cube2.setLength(10);

        Console.WriteLine("Volume of cube1 is {0}", cube1.getVolume());
        Console.WriteLine("Volume of cube2 is {0}", cube2.getVolume());
    }
}
```

Microsoft Visual Studio Debug Console

```
Volume of cube1 is 110
Volume of cube2 is 220
```

In our next example we define a class called Person with an uppercase P. (Figure 7.3) Be certain to create that class up at the top of your code where we created our previous Cube class. In fact, most professional programmers define their classes in a separate file and then associate them with the executable code via a solution. We will keep it simple here and keep all the syntax in a single file.

Figure 7.3: Person class

```
1 reference
class Person
{
    private string fn, ln, g;
    private int a, w, h;

    0 references
    public Person(string firstname, string lastname, string gender,
                  int age, int weight, int height)
    {
        Console.WriteLine("Person object is being created in memory"); // normally not included here
        fn = firstname;
        ln = lastname;
        g = gender;
        a = age;
        w = weight;
        h = height;
    }
}
```

What I want you to notice about this class is the public Person method with a bunch of input parameters. This is known as a parameterized constructor. It is nice because it explicitly tells intellisense what properties are associated with the Person class. When a user instantiates an object of this class, they know exactly what properties need to be provided. We'll see that here in just a moment. For now, please note the Console.WriteLine, under normal conditions you would not include this in your code. We included it here to demonstrate when memory is actually allocated during the instantiation process.

Our next image shows some additional methods associated with our Person class. (Figure 7.4)

Figure 7.4: Person class Part 2

```
private string fn, ln, g;
private int a;
private double w, h;

1 reference
public Person(string firstname, string lastname, string gender,
               int age, double weight, double height)
{
    Console.WriteLine("Person object is being created in memory"); // normally not included here
    fn = firstname;
    ln = lastname;
    g = gender;
    a = age;
    w = weight;
    h = height;
}

2 references
public string getFirstName() { return fn; }
0 references
public string getLastName() { return ln; }
0 references
public string getGender() { return g; }
0 references
public int getAge() { return a; }
0 references
public int getWeight() { return w; }
0 references
public int getHeight() { return h; }
```

To complete our Person class definition, I am going to add two additional methods. (Figure 7.5) The first is named `isAFatty` and it returns a boolean data type equal to `true` if weight divided by height is greater than 2.25. If weight divided by height is less than or equal to 2.25 this method returns a value of `false`. Also, please note that this method is private to the class definition. This technique is known as encapsulation. In many applications there is logic and/or code which cannot be shared with users or even other programmers, so we hide it via these private methods. Please note that the `failsBMI` method does nothing more than call the `isAFatty` method but it is public so other programmers and the outside world can have access to it.

Figure 7.5: Person class Part 3

```
1 reference
public Person(string firstname, string lastname, string gender,
               int age, double weight, double height)
{
    Console.WriteLine("Person object is being created in memory"); // normally not included here
    fn = firstname;
    ln = lastname;
    g = gender;
    a = age;
    w = weight;
    h = height;
}

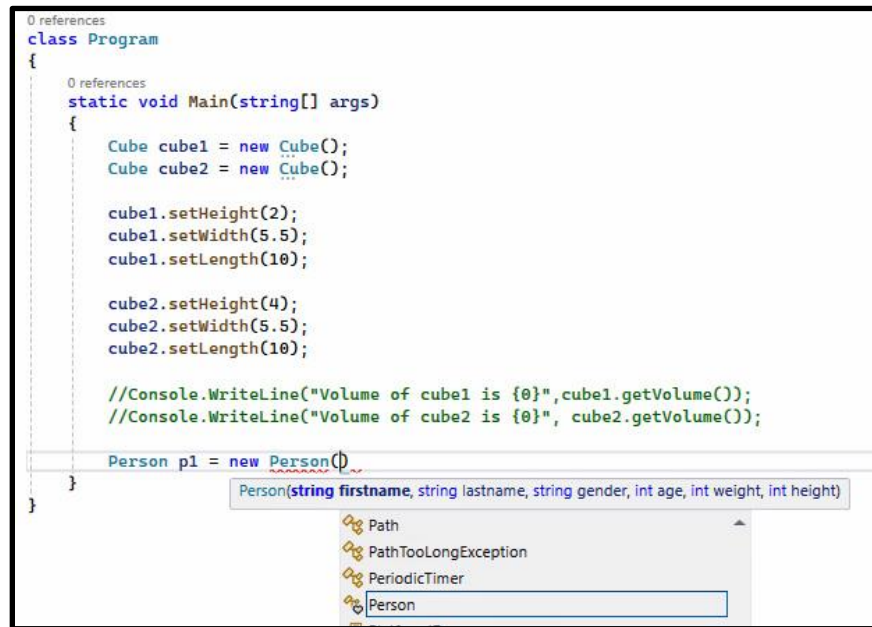
2 references
public string getFirstName() { return fn; }
0 references
public string getLastName() { return ln; }
0 references
public string getGender() { return g; }
0 references
public int getAge() { return a; }
0 references
public double getWeight() { return w; }
0 references
public double getHeight() { return h; }

1 reference
private bool isAFatty()
{
    if (w / h > 2.25) return true;
    else return false;
}

1 reference
public bool failsBMI()
{ return isAFatty(); }
```

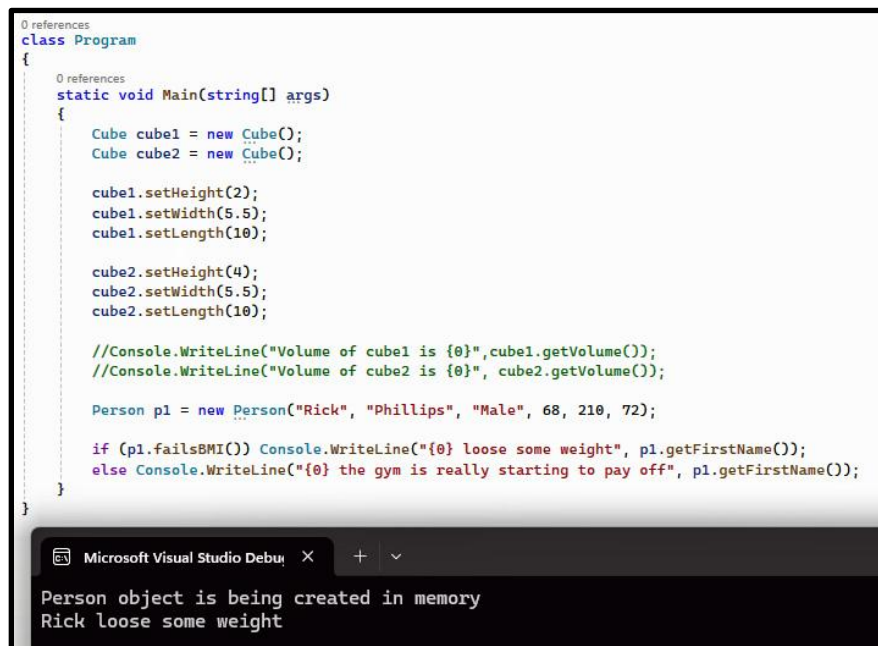
In our next image I am back in the Main block of code instantiating a Person class object. (Figure 7.6) Check out how intellisense is telling me exactly what properties I need to create and of what data type. This of course is the big advantage of creating a parameterized constructor. What is the bad news? Our example here is super simple, when we get much more complex class definitions later on in this course a parameterized constructor really isn't very feasible.

Figure 7.6: Parameterized Constructor Method



In this next image we go ahead and complete the instantiation of our p1 Person object and then play a little joke on myself with the if else code. (Figure 7.7)

Figure 7.7: Person class Instantiation 1



```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Cube cube1 = new Cube();
        Cube cube2 = new Cube();

        cube1.setHeight(2);
        cube1.setWidth(5.5);
        cube1.setLength(10);

        cube2.setHeight(4);
        cube2.setWidth(5.5);
        cube2.setLength(10);

        //Console.WriteLine("Volume of cube1 is {0}", cube1.getVolume());
        //Console.WriteLine("Volume of cube2 is {0}", cube2.getVolume());

        Person p1 = new Person("Rick", "Phillips", "Male", 68, 210, 72);

        if (p1.failsBMI()) Console.WriteLine("{0} loose some weight", p1.getFirstName());
        else Console.WriteLine("{0} the gym is really starting to pay off", p1.getFirstName());
    }
}
```

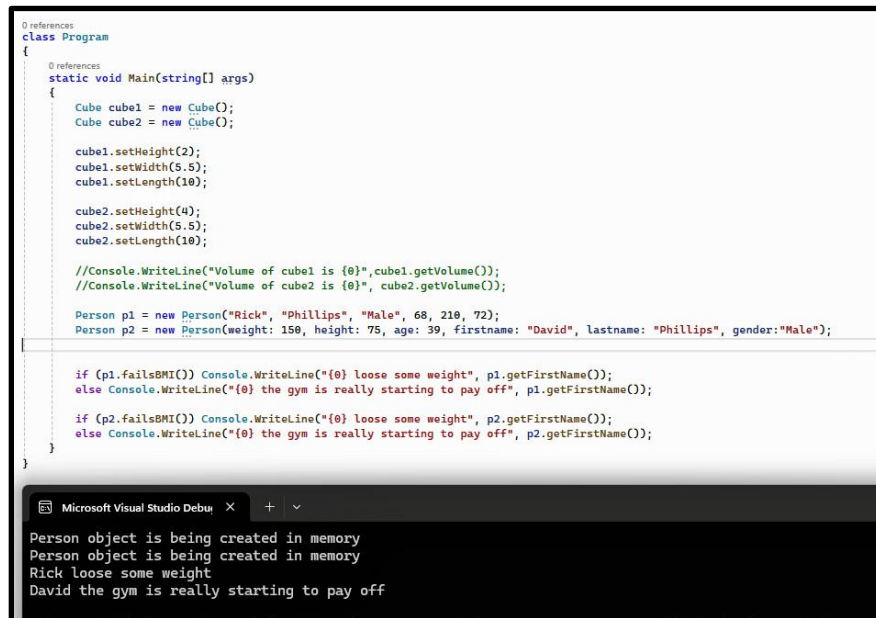
Microsoft Visual Studio Debug Console

Person object is being created in memory
Rick loose some weight

As an aside, can you tell me what our `isAFatty` routine would have returned with this same Person object if we had declared the weight and height properties to be integer data types? If you said false, then you are correct. It would have divided 210 by 72 and returned a value of 2 because it would have truncated the result to an integer. What did we learn? ALWAYS make your numeric properties doubles instead of integers if you intend to do arithmetic on those properties.

In our next image we create a second instance of the Person class this time explicitly specifying the parameter names. (Figure 7.8) Please note the use of the colon and the fact that you do not need to worry about the parameter order using this syntax.

Figure 7.8: Explicit Parameter Specification



```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Cube cube1 = new Cube();
        Cube cube2 = new Cube();

        cube1.setHeight(2);
        cube1.setWidth(5.5);
        cube1.setLength(10);

        cube2.setHeight(4);
        cube2.setWidth(5.5);
        cube2.setLength(10);

        //Console.WriteLine("Volume of cube1 is {0}", cube1.getVolume());
        //Console.WriteLine("Volume of cube2 is {0}", cube2.getVolume());

        Person p1 = new Person("Rick", "Phillips", "Male", 68, 210, 72);
        Person p2 = new Person(weight: 150, height: 75, age: 39, firstname: "David", lastname: "Phillips", gender: "Male");

        if (p1.failsBMI()) Console.WriteLine("{0} loose some weight", p1.getFirstName());
        else Console.WriteLine("{0} the gym is really starting to pay off", p1.getFirstName());

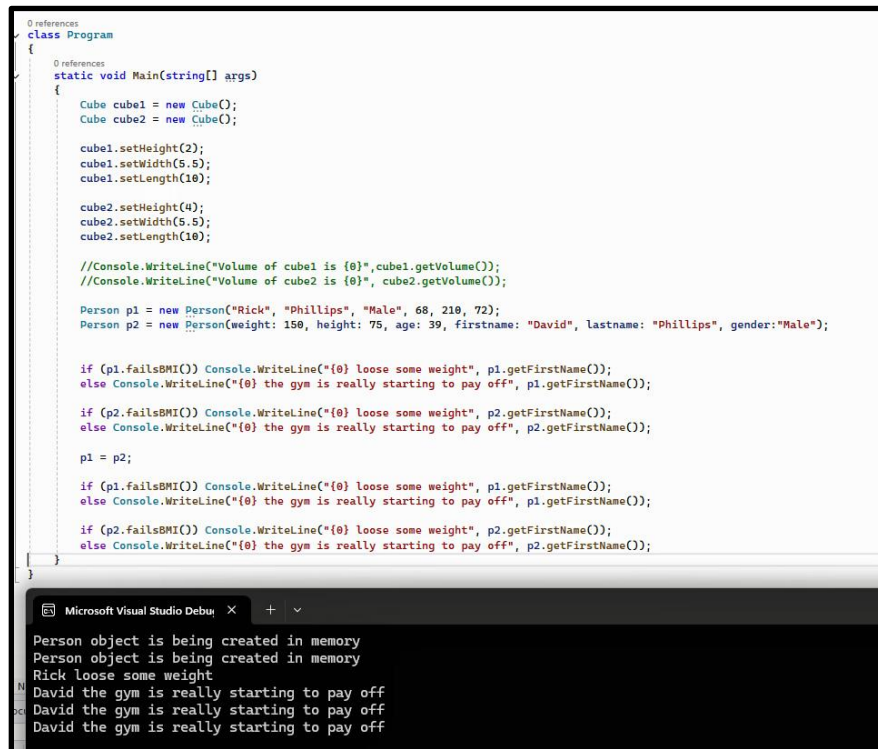
        if (p2.failsBMI()) Console.WriteLine("{0} loose some weight", p2.getFirstName());
        else Console.WriteLine("{0} the gym is really starting to pay off", p2.getFirstName());
    }
}
```

Microsoft Visual Studio Debug Console

```
Person object is being created in memory
Person object is being created in memory
Rick loose some weight
David the gym is really starting to pay off
```

We conclude this chapter with the example shown here. (Figure 7.9) Classes are stored in memory by reference so when we set our p1 instance to the value of p2 we are really pointing p1 at the p2 memory location. This differs from the structures we learned about in the previous chapter which are stored by value. This distinction will become very important as we start passing classes around between various subroutines in subsequent chapters.

Figure 7.9: Classes Stored by Reference



```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Cube cube1 = new Cube();
        Cube cube2 = new Cube();

        cube1.setHeight(2);
        cube1.setWidth(5.5);
        cube1.setLength(10);

        cube2.setHeight(4);
        cube2.setWidth(5.5);
        cube2.setLength(10);

        //Console.WriteLine("Volume of cube1 is {0}", cube1.getVolume());
        //Console.WriteLine("Volume of cube2 is {0}", cube2.getVolume());

        Person p1 = new Person("Rick", "Phillips", "Male", 68, 210, 72);
        Person p2 = new Person(weight: 150, height: 75, age: 39, firstname: "David", lastname: "Phillips", gender: "Male");

        if (p1.failsBMI()) Console.WriteLine("{0} loose some weight", p1.getFirstName());
        else Console.WriteLine("{0} the gym is really starting to pay off", p1.getFirstName());

        if (p2.failsBMI()) Console.WriteLine("{0} loose some weight", p2.getFirstName());
        else Console.WriteLine("{0} the gym is really starting to pay off", p2.getFirstName());

        p1 = p2;

        if (p1.failsBMI()) Console.WriteLine("{0} loose some weight", p1.getFirstName());
        else Console.WriteLine("{0} the gym is really starting to pay off", p1.getFirstName());

        if (p2.failsBMI()) Console.WriteLine("{0} loose some weight", p2.getFirstName());
        else Console.WriteLine("{0} the gym is really starting to pay off", p2.getFirstName());
    }
}
```

Microsoft Visual Studio Debug Console Output:

```
Person object is being created in memory
Person object is being created in memory
Rick loose some weight
David the gym is really starting to pay off
David the gym is really starting to pay off
David the gym is really starting to pay off
```

So, what did we learn about classes in this chapter?

- We learned how to define a class along with its associated properties and methods.
- We learned how to instantiate a class with and without a parameter list.
- We learned how to create a parameterized constructor method.
- We learned that Rick is probably a little overweight... ha, ha, ha.

Okay that concludes our first lesson in classes. We continue to look at classes in the next lesson wherein we get into inheritance, powerful stuff.